



DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi, Approved by AICTE and UGC, Accredited by NAAC with 'A' grade)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111



DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)

Alternate Assessment Tool - 2 “Scenario Investigation”

Advanced Jenkins Pipeline Architecture: Optimizing Builds, Tests and Deployment Processes for Enterprise Applications

Report submitted for the subject
INTRODUCTION TO AGILE-DEVOPS INTEGRATION – 22IS673

Submitted by

DIKSHITH M (1DS23IS044)

SEMESTER: VI GROUP – A08

Under the Guidance of Dr. Reshma J Associate Professor
Department of Information Science and Engineering DSCE, Bengaluru

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI-590018, KARNATKA 2025-26

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi, Approved by
AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)



CERTIFICATE

This is to certify that the Alternate Assessment Tool (AAT) - 2 submitted as part of **INTRODUCTION TO AGILE-DEVOPS INTEGRATION (22IS673)** is a bonafide work carried out by **Dikshith M (1DS23IS044)** as 10-mark component in partial fulfillment for the 6th Semester of Bachelor of Engineering in Information Science & Engineering of the Visvesvaraya Technological University, Belagavi during the year 2025-26. The AAT report has been approved as it satisfies the academic requirements prescribed for the Bachelor of Engineering degree.

Signature of Faculty
[Dr.Reshma J]

Signature of HOD
[Dr.Annapurna P Patil]

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi, Approved by
AICTE and UGC, Accredited by NAAC with 'A' grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

(Accredited by NBA Tier 1: 2025-2028)



DECLARATION

I declare that I abide by the ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. The work submitted in this report of INTRODUCTION TO AGILE-DEVOPS INTEGRATION – 22IS673, VI Semester BE, ISE has been compiled by referring to the relevant online and offline resources to the best of my understanding and in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Information Science & Engineering, at Dayananda Sagar College of Engineering, an autonomous institution affiliated to VTU, Belagavi during the academic year 2025-26.

I hereby declare that the same has not been submitted in part or full for other academic purposes.

Dikshith M (1DS23IS044)

Place: Bangalore

Date:

Contents

Introduction	5
Scenario Overview	6
Parallel Execution Strategies	7
Managing Dependencies	9
Resource Management	11
Test Optimization	12
Multi-Environment Deployment Strategy	13
Sample Pipeline Structure	14
Conclusion	15
References	16

1. Introduction

Running a Jenkins pipeline for a single codebase is straightforward enough. The real work starts when the application has a frontend and a backend that are largely independent but still need to be integrated, tested together, and shipped to multiple environments without the pipeline becoming a bottleneck. Getting this right requires deliberate choices about what runs in parallel, how dependencies between stages are enforced, and where compute resources are allocated.

This report examines how to structure such a pipeline, using the scenario of a web application with separate frontend and backend components that must pass through development, staging, and production environments. The focus is on practical strategies: how to split work across parallel stages, how to handle situations where one component depends on the other, and how to avoid the common trap of pipelines that nominally run jobs in parallel but still take forever because of poor agent configuration or unnecessary sequential bottlenecks.

2. Scenario Overview

The application in question has a frontend built with a JavaScript framework (React or Vue, the specifics do not change the pipeline logic much) and a backend written in a server-side language such as Node.js or Java. Both components have their own test suites, build artifacts, and deployment procedures. Three environments exist: development, which receives every commit to main branches; staging, which mirrors production and runs integration and end-to-end tests; and production, which requires a manual approval gate before deployment.

The naive approach would be to run all frontend work sequentially, then all backend work, then deploy. That approach works but is slow. A commit that breaks only the frontend unit tests still causes the backend build to wait. When teams are pushing several times a day, that wait time adds up quickly.

The better approach is to treat the frontend and backend as parallel streams that merge only at the points where they genuinely depend on each other, then enforce those merge points explicitly in the pipeline definition.

3. Parallel Execution Strategies

3.1 Splitting the Pipeline into Independent Lanes

Jenkins supports parallel stages natively in Declarative Pipeline syntax. The first practical split is at the build and unit test phase. Frontend compilation and backend compilation are entirely independent at this point, so they can run simultaneously on separate agents. A simplified view of this looks like the following:

```
stage('Build and Unit Test') {
    parallel {
        stage('Frontend') {
            agent { label 'node-agent' }
            steps {
                sh 'npm ci'
                sh 'npm run build'
                sh 'npm test -- --ci'
            }
        }
        stage('Backend') {
            agent { label 'java-agent' }
            steps {
                sh 'mvn clean package -DskipTests=false'
            }
        }
    }
}
```

Running these in parallel on separate labeled agents cuts the build time roughly in half, assuming agents are available. The key is labeling agents by capability (node-agent for JavaScript tooling, java-agent for JVM-based builds) so Jenkins does not assign a frontend job to an agent that lacks npm or vice versa.

3.2 Handling the Integration Boundary

After both builds pass their unit tests, integration testing requires both artifacts. This is where the parallel lanes must merge. Jenkins handles this cleanly because a parallel block only completes when all branches inside it complete. Stages defined after the parallel block run sequentially, with all artifacts from both branches available.

The integration test stage spins up both services, usually in Docker containers, and runs tests against the combined system. This stage should run on an agent with Docker access. Importantly, it only runs if both the frontend and backend stages passed, which is the default Jenkins behavior when using parallel blocks without the failFast option disabled.

3.3 Failing Fast Without Blocking Unrelated Work

Jenkins parallel stages support a failFast option. When set to true, a failure in one branch immediately cancels the other branches. For the build and unit test phase, this is usually desirable: if the backend build fails, there is no point waiting for the frontend to finish before showing the developer a failure. However, there are cases where you want both results even on failure, particularly for reporting. The choice depends on team preference and how quickly developers need feedback.

A good middle ground is to set failFast true for the build phase but false for lint and static analysis phases, where both frontend and backend results are useful to developers independently.

4. Managing Dependencies

4.1 Package Dependencies and Caching

Dependency installation is often the slowest part of a frontend build. npm ci on a cold cache can take minutes. Jenkins does not cache node_modules between builds by default, so every build starts fresh. There are two practical solutions:

- Mount a shared Docker volume or NFS path that contains a cached npm or Maven local repository. Each build checks the cache before downloading from the internet. This works well for teams with stable dependency graphs.
- Use Jenkins artifact caching plugins such as the Shared Library approach with stash and unstash, or leverage the native Maven local repository cache on persistent agents. For npm, the --prefer-offline flag combined with a cached modules directory reduces install time significantly.

For the backend with Maven or Gradle, the local .m2 or .gradle cache on the agent is the simplest approach. If agents are ephemeral (Docker-based), that cache needs to be mounted from a persistent volume.

4.2 Artifact Passing Between Stages

When parallel stages produce artifacts that subsequent stages need, those artifacts must be explicitly passed. Jenkins provides stash and unstash for this. After the frontend build completes, the compiled assets are stashed. After the backend build completes, the JAR or Docker image is stashed or pushed to a registry. The integration test stage unstashes or pulls both before running.

For larger artifacts, pushing to a registry (Docker Hub, Nexus, Artifactory) rather than using stash is more reliable. stash is essentially a tar archive stored on the Jenkins controller, which can become a bottleneck for large artifacts or high-frequency pipelines.

4.3 Environment-Specific Configuration

Frontend and backend configuration that varies per environment (API URLs, feature flags, database endpoints) should not be baked into the build artifact. The artifact should be environment-agnostic, with configuration injected at deployment time. Jenkins credentials and environment variables handle this:

- Use Jenkins Credentials Binding to inject secrets (database passwords, API keys) only at deployment stages, not during the build.

- Use environment-specific Jenkinsfile parameters or separate Jenkinsfile branches for configurations that differ fundamentally between environments.
- For Kubernetes-based deployments, ConfigMaps and Secrets handle per-environment configuration outside the pipeline entirely.

5. Resource Management

5.1 Agent Allocation and Labeling

Parallel stages are only as fast as agent availability allows. If there is only one agent in the pool and two parallel stages need agents simultaneously, they end up running sequentially anyway. Capacity planning for parallel pipelines requires at minimum as many available agents as the maximum number of concurrent stages.

Labeling agents by capability is more practical than maintaining dedicated per-project agents. A label like build-agent covers any agent with general build tools, while specialized labels like docker-agent or gpu-agent handle specific requirements. Jenkins matches stages to agents by label, and the build queue handles contention when demand exceeds supply.

5.2 Docker-Based Agents

Jenkins' Docker Pipeline plugin allows stages to specify Docker images directly as agent definitions. This eliminates the need to pre-configure physical or virtual agents with project-specific tooling:

```
stage('Frontend Build') {  
    agent {  
        docker { image 'node:18-alpine' }  
    }  
    steps {  
        sh 'npm ci && npm run build'  
    }  
}
```

Docker-based agents start fresh for each build, eliminating the contamination risk where one build leaves files that affect the next. The trade-off is startup time: pulling a Docker image on each run adds latency. This is mitigated by pre-pulling commonly used images on agent hosts, or using a local Docker registry that mirrors upstream images.

5.3 Throttling and Queue Management

Without throttling, a team with aggressive CI practices can saturate the agent pool. The Jenkins Throttle Concurrent Builds plugin allows limits on how many builds of a given pipeline or category can run simultaneously. For the production deployment stage specifically, throttling to one concurrent execution prevents simultaneous deployments that could cause race conditions or partial rollouts.

For staging deployments, a reasonable limit is two or three concurrent builds. Development deployments can often run without throttling since developers expect fast feedback on that environment.

6. Test Optimization

6.1 Test Categorization and Stage Assignment

Not all tests need to run on every commit. A practical categorization for this pipeline is:

- Unit tests: Run in the parallel build stage, directly alongside the build. Fast, no external dependencies. Both frontend and backend unit tests run in parallel with their respective builds.
- Integration tests: Run after both components are built and deployed to a test environment. These take longer and require infrastructure, so they run once per merged artifact pair rather than for every individual component build.
- End-to-end tests: Run only on the staging environment before production promotion. These are the slowest and most brittle tests. Running them on every commit to development creates noise.
- Smoke tests: Run after production deployment to confirm the deployment itself did not break basic functionality. Keep these very fast, under two minutes.

6.2 Parallelizing Tests Within a Stage

Within a single stage, test execution can also be parallelized. Jest (frontend) supports running test suites across worker threads with the `--maxWorkers` flag. Maven Surefire (backend) supports parallel test execution with the `forkCount` configuration. On agents with multiple CPU cores, these settings can cut test time significantly without any pipeline structural changes.

For integration tests that involve multiple services, consider running test groups against different subsets of functionality in parallel, each with its own isolated Docker Compose environment. This requires more infrastructure but scales well as the test suite grows.

7. Multi-Environment Deployment Strategy

The pipeline promotes artifacts across environments in order: development, then staging, then production. Each promotion involves deploying the same artifact (the one built in the build stage) rather than rebuilding. Rebuilding per environment is a common mistake that introduces inconsistency: the artifact deployed to staging is not literally the same binary that was tested in development.

Environment promotion gates work as follows:

- Development: Automatic deployment on every successful build of the main branch. No approval required. Frontend and backend deploy in parallel.
- Staging: Automatic deployment after development smoke tests pass. Integration and end-to-end tests run here. If they pass, the pipeline marks the build as eligible for production.
- Production: Manual input step requiring explicit approval from a defined group. Jenkins input step holds the pipeline and notifies approvers via email or Slack webhook. Deployment runs sequentially after approval, with rollback logic if smoke tests fail post-deployment.

Rolling back in production means deploying the previous known-good artifact, not re-running the pipeline from scratch. The pipeline should store the last successful artifact reference (Docker image tag or S3 object version) and expose a rollback parameter that bypasses the full pipeline and deploys that artifact directly.

8. Sample Pipeline Structure

A representative Declarative Pipeline for this scenario has the following stage structure:

```

pipeline {
  agent none
  stages {
    stage('Build & Unit Test') {
      parallel {
        stage('Frontend') { agent { docker { image 'node:18' } }
          steps { sh 'npm ci && npm run build && npm test -- --ci' }
          post { always { junit 'test-results/*.xml' } }
        }
        stage('Backend') { agent { docker { image 'maven:3.9-eclipse-temurin-
17' } }
          steps { sh 'mvn clean package' }
          post { always { junit 'target/surefire-reports/*.xml' } }
        }
      }
    }
    stage('Deploy to Dev') {
      parallel {
        stage('Deploy Frontend Dev') { ... }
        stage('Deploy Backend Dev') { ... }
      }
    }
    stage('Integration Tests') { agent { label 'docker-agent' }
      steps { sh './run-integration-tests.sh' }
    }
    stage('Deploy to Staging') { ... }
    stage('E2E Tests') { ... }
    stage('Approval') {
      steps { input message: 'Deploy to production?', submitter: 'ops-team' }
    }
    stage('Deploy to Production') {
      parallel {
        stage('Deploy Frontend Prod') { ... }
        stage('Deploy Backend Prod') { ... }
      }
    }
    stage('Production Smoke Test') { ... }
  }
}

```

This structure makes the dependency graph explicit. Anyone reading the Jenkinsfile can see exactly where parallel work merges, where tests run, and where human gates exist. That clarity is worth more than most optimizations in practice, because unclear pipelines tend to accumulate workarounds that eventually cause incidents.

9. Conclusion

Optimizing a Jenkins pipeline for a multi-component web application comes down to a few concrete decisions: split independent work into parallel lanes, merge those lanes only where dependencies require it, cache what is expensive to rebuild, and deploy the same artifact across environments rather than rebuilding per environment.

Parallel execution between frontend and backend stages is the highest-leverage change for most teams. The savings are immediate and require no architectural changes to the application itself. Dependency caching and Docker-based agents address the next tier of latency. Beyond that, test categorization, ensuring fast tests run early and slow tests run only where necessary, keeps the pipeline from becoming a bottleneck as the test suite grows.

The manual approval gate before production remains the right call for most applications, regardless of how automated everything else is. Automation handles repetition well. Human judgment handles the edge cases that automation has not seen yet. A pipeline that automates everything up to that gate, and makes the gate easy to act on, balances speed with prudence.

10. References

- [1] Smart, J. F. (2011). Jenkins: The Definitive Guide. O'Reilly Media.
- [2] Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
- [3] Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.
- [4] Jenkins Documentation. (2024). Declarative Pipeline Syntax.
- [5] Jenkins Documentation. (2024). Using Docker with Pipeline.
- [6] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- [7] Richardson, C. (2018). Microservices Patterns. Manning Publications.
- [8] Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.